Weniger Compute bei gleicher Modellqualität

Forschungsstand 2024 bis 2026 und eigene Beiträge

OVERFLOW / Leon Pelzmann / PoeziAI

05.10.2026

Hinweis

Das Dokument trennt den Stand der Technik von den eigenen Beiträgen. Der erste Teil ordnet die Literatur gegen die Randbedingungen des Projekts ein; der zweite Teil dokumentiert die umgesetzten Verfahren, deren Messung und formale Grundlagen.

● Eigener Beitrag ● Stand der Technik

Inhaltsverzeichnis

1 Randbedingungen 2

2 Stand der Technik 2

2.1 Verlustfreie Gewichts-Kompression 2

2.2 Aktivierungs-Sparsität 3

2.3 Offloading-Systeme 3

2.4 Spekulatives Decoding 3

2.5 KV-Cache-Kompression 4

2.6 Adaptive Rechentiefe 4

2.7 MoE statt dichtem Modell 4

2.8 Domänenspezialisierung 4

3 Eigene Beiträge 5

3.1 Messungen 5

3.2 Umgesetzte Verfahren 5

3.3 Neuheitsgrad 6

3.4 Formale Grundlagen 6

4 Experiment: Domänenspezifische Sparsität 8

5 Synthese 8

6 Forschungsagenda 8

7 Zusammenfassung 9

8 Quellen 10

1 Randbedingungen

● Eigener Beitrag

Zwei Vorgaben schränken die anwendbare Literatur ein: keine Quantisierung (sie senkt die Modellqualität) und keine eigenen CUDA-Kernel. Die meisten veröffentlichten GPU-Beschleunigungen setzen eigene Kernel voraus. In reinem Rust bleiben die Orchestrierungs-Ebene und der CPU-Pfad.

Faktor	Wert
Modell	dichtes Qwen2.5-Coder (7B, perspektivisch 14B/32B), SwiGLU/SiLU, kein MoE
GPU	RTX 3060, 12 GB (~10 GB nutzbar mit Desktop)
CPU	~37x langsamer pro Token als die GPU
Präzision	voll, keine Quantisierung
Umsetzung	Rust, keine eigenen CUDA-Kernel
Workload	agentisch, Domäne Security und Code

Zielkonflikt: Von den drei Eigenschaften passt-in-VRAM, volle Qualität und volle Geschwindigkeit sind bei einem dichten Modell, das größer als der VRAM ist, nur zwei zugleich erreichbar, solange jedes Gewicht bei jedem Token gelesen wird. Die betrachteten Verfahren zielen darauf, diese Annahme aufzuheben.

2 Stand der Technik

● Stand der Technik

2.1 Verlustfreie Gewichts-Kompression

Das Modell verlustfrei kleiner speichern und zur Laufzeit entpacken; die Ausgabe bleibt bit-identisch.

- **DFloat11** (NeurIPS 2025). Huffman-Codierung der BF16-Exponenten, ~30% kleiner bei bit-identischer Ausgabe. Übertragen: 15 GB werden zu ~10,5 GB. arxiv 2504.11651
- **Approaching Shannon Bound** (2026). Kompression näher am informationstheoretischen Optimum. arxiv 2606.15789
- **NeuZip**. Frühere verlustfreie Kompression als Vergleichsbasis.

> **Bewertung: bedingt**
> Das Entpacken erfordert einen GPU-Decode-Kernel (Konflikt mit der Kernel-Vorgabe). DFloat11 zielt auf BF16; die vorhandenen Gewichte sind bereits Q8, die Entropie-Reserve ist geringer.

2.2 Aktivierungs-Sparsität

Pro Token tragen die meisten FFN-Neuronen wenig bei. Die Herausforderung ist die Nutzung bei SwiGLU/SiLU ohne harte Nullen.

- **TEAL** (ICLR 2025). Magnitude-Sparsität ohne Retraining, 40 bis 50% modellweit, Faktor 1,5 bis 1,8. arxiv 2408.14690
- **CATS**. Training-freie Sparsität für SwiGLU, Magnitude-Pruning am Wgate-Ausgang.
- **SparseInfer**. Training-freie Vorhersage des aktiven Satzes. arxiv 2411.12692
- **Polar Sparsity** (2025). Durchsatzorientierte Variante. arxiv 2505.14884
- **DejaVu**, **PowerInfer**: fester Hot-Satz, wirksam nur bei ReLU-Modellen.

> **Bewertung: wirksam, kernel-abhängig**
>
> 40 bis 50% der FFN-Rechnung sind verlustfrei streichbar; die Einsparung erfordert einen Sparse-Kernel. In reinem Rust ist der Nutzen auf den CPU-Offload-Pfad beschränkt.

2.3 Offloading-Systeme

Aufteilung des Modells zwischen GPU und CPU/RAM, verlustfrei.

- **PowerInfer**. GPU-CPU-Hybrid mit Sparsität und Offline-Placement, bis Faktor 11,69 über llama.cpp. PowerInfer
- **llama.cpp**. Ganze Layer auf GPU, Rest auf CPU; begrenzt durch PCIe und CPU.
- **DynamicInfer** (ICLR 2026), **PIPO**, **FlexInfer**, **NEO**, **FlexGen**. Dynamisches Placement, Transfer-Compute-Overlap. PIPO, NEO

> **Bewertung: anwendbar, reines Rust**
>
> Fit-Weg über ganze Layer, bestehende Kernel wiederverwendbar. Der Faktor 11 von PowerInfer setzt ReLU und eigene Kernel voraus und ist nicht übertragbar.

2.4 Spekulatives Decoding

Mehrere Token je Durchlauf erzeugen und gemeinsam prüfen; bei Temperatur 0 verlustfrei.

- **Prompt-Lookup**, **Lookahead**. Kein zweites Modell; Entwürfe aus dem Kontext.
- **Medusa**. Draft-Heads am Zielmodell.
- **EAGLE-2/3**. Stand der Technik, Faktor ~1,98. EAGLE-3
- Survey EMNLP-Findings 2025. Survey

> **Bewertung: umgesetzt**
>
> Die training-freie Variante ist implementiert (siehe Eigene Beiträge).

2.5 KV-Cache-Kompression

Betrifft den mit der Kontextlänge wachsenden KV-Cache, nicht die Gewichte.

- **StreamingLLM**, **H2O**, **SnapKV**. Auswahl wichtiger KV-Positionen.
- **PyramidKV**. Schichtweise unterschiedliche Kürzung. PyramidKV
- **KIVI**. KV-Quantisierung. **THINK**. Query-abhängiges Pruning.

> **Bewertung: später relevant**
> Für den VRAM-Bedarf der Gewichte nicht einschlägig.

2.6 Adaptive Rechentiefe

- **Mixture-of-Depths** (2024). Nicht jedes Token durchläuft jeden Layer.
- **Mixture-of-Recursions** (2025). Token-abhängige Rekursionstiefe. MoR
- **Early-Exit**. Vorzeitiger Abbruch bei hinreichender Sicherheit.

> **Bewertung: Architektur oder Training**
> Erfordert ein anders trainiertes Modell.

2.7 MoE statt dichtem Modell

15 GB Gesamtparameter, aber nur ~3 GB pro Token aktiv; der aktive Satz passt in den VRAM.

- **MoE-Infinity**. Experten ausgelagert, aktive Experten resident. MoE-Infinity
- **DALI**, **HybriMoE**, **fMoE**. Workload-abhängiges Experten-Offloading. DALI

> **Bewertung: strategisch**
> Setzt einen Wechsel vom dichten Qwen2.5 voraus.

2.8 Domänenspezialisierung

Ein kleinerer, stabiler aktiver Satz für eine enge Domäne ist bereits untersucht.

- **SEAP**. Training-freies Pruning task-spezifischer Neuronen. SEAP
- **DieT**. Profilierung der Aktivierungsmagnituden über Tasks, globale Maske. Diet Your LLM
- **Half the Experts, All the Code**, **TrimLLM**. Domänen-Pruning für Code. Half the Experts

> **Bewertung: nicht neu**
> Der Ansatz existiert. Beitrag dieses Projekts ist der Nachweis, dass er für das vorliegende Modell nicht trägt (siehe Experiment).

3 Eigene Beiträge

● Eigener Beitrag

3.1 Messungen

Erhoben am Modell overflow_wt-7B mit den Sonden `poezi sparsity`, `masktest` und `skew`:

- FFN: ~87% der Gewichte je Block.
- Aktivierungs-Sparsität: ~4,5% der FFN-Neuronen über 5% des Maximums, ~35% über 1%.
- Oracle-Maskierung: die oberen 50% behalten ist bit-identisch; darunter sinkt die Top-1-Übereinstimmung (25% ergibt 90%, 10% ergibt 78%).
- Statischer Skew: die 25% häufigsten Neuronen decken ~39% der Aktivierungen; SiLU ergibt kein Power-Law.

3.2 Umgesetzte Verfahren

In Rust implementiert (mit Ausnahme der zuvor geschriebenen Spec-Decode-Kernel), auf der RTX 3060 vermessen:

Verfahren	Ergebnis
Speculative Decoding	verlustfrei, Faktor 1,67 auf Code/Agent-Reuse
AVX2-CPU-Matmul	Faktor 4 im Kernel (5 auf 19,5 GB/s), Faktor 2 im CPU-Forward
Whole-Layer-Offloading	verlustfrei, 15-GB-Modell läuft auf 12 GB
Sparse CPU-FFN (opt-in)	plus 13 bis 24% auf dem CPU-Pfad
Sparsity-aware Fine-Tuning	induziert g-Sparsität (35% auf 21%), Validierung negativ (siehe unten)

Offloading-Durchsatz nach CPU-Anteil (7B, 40 Token):

GPU-Layer	Token/s
28 von 28	5,7
20 von 28	3,6
14 von 28	2,9
7 von 28	2,4

3.3 Neuheitsgrad

Die zugrunde liegenden Verfahren sind publiziert. Neu sind ihre Kombination und einzelne Resultate unter den gegebenen Randbedingungen.

Bekannt, hier eigenständig umgesetzt: Prompt-Lookup Speculative Decoding, Whole-Layer-Offloading, AVX2-Matmul, CATS-artige FFN-Sparsität.

Nach derzeitigem Kenntnisstand in dieser Form nicht vorhanden und funktionsfähig:

1. Ein integrierter, gegen llama.cpp bit-identisch verifizierter Inferenz-Stack **ohne externe Inferenzbibliothek**, der Speculative Decoding, auto-dimensioniertes Whole-Layer-Offloading und opt-in CPU-Sparsität auf einer 12-GB-Consumer-GPU unter der Vorgabe keiner eigenen Kernel für die Orchestrierung vereint.
2. Batched, bit-identisches Quant-GEMM zur spekulativen Verifikation (Q8_0/Q4_K/Q6_K), gemessen token-identisch zum Einzel-Token-Pfad.
3. Empirisches Resultat: Für Qwen2.5-Coder ist der aktive FFN-Satz nicht domänenspezifisch (Differenz 3 Prozentpunkte); statisches Domänen-Offloading trägt nicht. Ein in dieser Form nicht publiziertes Negativresultat.
4. Offene Richtung mit negativem Zwischenergebnis: sparsity-aware Fine-Tuning senkte zwar den aktiven FFN-Anteil (35% auf 21% über 100 Schritte, Textprobe kohärent), aber die Validierung widerlegt den Nutzen: das trainierte Modell verträgt die ausnutzbare CATS-Sparsität schlechter als die Basis (keep 25%: 62% gegen 90% Übereinstimmung). Ursache vermutlich, dass die Strafe auf der post-SwiGLU-Größe g saß, der ausnutzbare Schwellenwert aber auf silu(gate); dazu Overfitting auf 73 Beispielen. Korrigierter Versuch (Strafe direkt auf silu(gate)) steht aus.

> **Hinweis**
> Kein algorithmischer Durchbruch. Der Beitrag liegt in der Integration, der bit-genauen Verifikation und den Messergebnissen für ein Regime (schwache GPU, langsame CPU, keine eigenen Kernel, enger Agent), das in der Literatur kaum abgedeckt ist.

3.4 Formale Grundlagen

Satz 1 — Verlustfreiheit des spekulativen Decodings. Behauptung. Bei Temperatur 0 erzeugt das Prompt-Lookup-Verfahren dieselbe Token-Folge wie gewöhnliches Greedy-Decoding.

Sei $z_t \in \mathbb{R}^V$ der Logit-Vektor des Forward-Passes auf dem Präfix $x_{1..t-1}$ und $x_t = \mathrm{argmax}_v z_t(v)$ die Greedy-Wahl. Das Verfahren bildet einen Entwurf $d_1, \ldots, d_m$, berechnet in einem Durchlauf die Logits $z^{(0)}, \ldots, z^{(m)}$ an den Entwurfspositionen und akzeptiert d_j genau dann, wenn

$$d_j = \mathrm{argmax}_v z^{(j-1)}(v).$$

Lemma (Bit-Identität). Der batched Kernel berechnet für jedes Paar aus Zeile und Token dieselbe Summe in derselben Reihenfolge wie der Einzel-Token-Kernel; die Reduktion ist identisch. Damit ist $z^{(j)}$ bit-gleich dem Logit-Vektor des sequentiellen Durchlaufs an gleicher Position nach den akzeptierten Token.

Beweis (Induktion über die akzeptierten Positionen). Ein akzeptiertes d_j erfüllt $d_j = \text{argmax}_v z^{(j-1)}(v)$ und ist nach dem Lemma die Greedy-Wahl. Das erste nicht akzeptierte bzw. das Anschluss-Token ist $\text{argmax}_v z^{(m)}(v)$, ebenfalls die Greedy-Wahl. Der Cache wird auf die akzeptierte Länge zurückgesetzt. Jedes ausgegebene Token stimmt folglich mit Greedy überein. **q.e.d.**

Satz 2 — Fehlerschranke beim Weglassen inaktiver Neuronen. Die FFN-Ausgabe ist eine gewichtete Summe der Neuron-Beiträge

$$y = \sum_{i=1}^{n} g_i w_i,$$

mit $w_i \in \mathbb{R}^d$ als i-ter Spalte von W_{down} und $g_i = \text{silu}(a_i)u_i$ als post-SwiGLU-Aktivierung. Bei Beibehaltung einer Menge S und Nullsetzen der übrigen g_i gilt

$$\|y - \hat{y}\| = \left\| \sum_{i \notin S} g_i w_i \right\| \leq \sum_{i \notin S} |g_i| \, \|w_i\| \leq \left(\max_i \|w_i\| \right) \sum_{i \notin S} |g_i| \, .$$

Folgerung. Bei festem $|S|$ wird die Schranke durch Weglassen der Neuronen mit kleinstem $|g_i|$ minimiert. Die Magnitude-Maskierung entspricht dem; die Messung ergibt Fehler 0 bei 50% Beibehaltung. **q.e.d.**

Satz 3 — Das Hoyer-Sparsitätsmaß. Die Trainingsstrafe minimiert $\frac{\|g\|_1}{\|g\|_2}$. Für $g \in \mathbb{R}^n, g \neq 0$ gilt

$$1 \leq \frac{\|g\|_1}{\|g\|_2} \leq \sqrt{n} \, .$$

Beweis. Die obere Schranke folgt aus Cauchy-Schwarz mit dem Einsvektor: $\|g\|_1 = \sum_i |g_i| \cdot 1 \leq \sqrt{\sum_i g_i^2} \sqrt{\sum_i 1} = \sqrt{n} \, \|g\|_2$, mit Gleichheit bei gleichen $|g_i|$ (dicht). Die untere Schranke folgt aus $\|g\|_2^2 = \sum_i g_i^2 \leq \left(\sum_i |g_i| \right)^2 = \|g\|_1^2$, mit Gleichheit bei höchstens einem von 0 verschiedenen g_i (maximal sparse).

Skaleninvarianz. Für $c > 0$ ist $\frac{\|cg\|_1}{\|cg\|_2} = \frac{\|g\|_1}{\|g\|_2}$. Das Maß erfasst die Form der Verteilung, nicht die Größe. Eine reine L_1-Strafe skaliert nur gleichmäßig und bewegt die relative Sparsität nicht. **q.e.d.**

Satz 4 — Verlustfreiheit des Layer-Offloadings. Der Forward-Pass ist eine Verkettung von Layer-Funktionen $h_1, \ldots, h_L$ mit Kopf p:

$$f(x) = p\left(h_L \left(\ldots h_{k+1} \left(h_k \left(\ldots h_1(x) \right) \right) \right) \right).$$

Beim Offloading berechnet die GPU $x' = h_k(\ldots h_1(x))$, überträgt den Zwischenzustand x' (Dimension d) auf die CPU, diese berechnet $x'' = h_L(\ldots h_{k+1}(x'))$, und die GPU liefert $p(x'')$. Da die Übertragung dem Zwischenzustand entspricht und die Komposition assoziativ ist, ergibt sich dasselbe $f(x)$ bis auf die Gleitkomma-Rundung zwischen den Geräten ($\sim 10^{-5}$), die die Greedy-Auswahl nicht ändert. Die Messung bei erzwungenem 14-von-28-Split ergibt token-identische Ausgabe zur vollen GPU. **q.e.d.**

4 Experiment: Domänenspezifische Sparsität

● Eigener Beitrag

Negatives Ergebnis

Ein Hot-Satz aus Security/Code-Prompts wurde gelernt und seine Abdeckung auf anderen Security/Code-Prompts gegen generischen Text gemessen (hot 25%): Domäne 36,1% gegen generisch 33,0%, Differenz 3,0 Prozentpunkte, nicht über der 39%-Baseline. Der Hot-Satz ist nicht domänenspezifisch. Statisches Domänen-Offloading trägt für das dichte SiLU-Modell nicht; die Hypothese ist widerlegt.

5 Synthese

● Eigener Beitrag

Die betrachteten Hebel sind publiziert. Die wenig untersuchte Lücke liegt im Regime, nicht in einer neuen Technik:

1. **Schwache GPU, langsame CPU und keine eigenen Kernel.** Ob die Verfahren der Literatur unter reiner Orchestrierung auf einer 3060 netto gewinnen ist offen.
2. **Einzelner, spezialisierter, fine-getunter Agent.** Die Domäne allein erzeugt keine ausnutzbare Struktur; sie müsste über sparsity-aware Fine-Tuning induziert werden.
3. **Gemessener kombinierter Stack** für dieses Gerät: Kompression, Sparsität und Spekulation zusammen, end-to-end.

Hinweis

Ein algorithmischer Durchbruch ist nicht zu erwarten. Realistisch ist ein Beitrag auf System- und Empirie-Ebene im Low-End- und Single-Agent-Regime.

6 Forschungsagenda

● Eigener Beitrag

1. **CPU-Matmul beschleunigen.** Grundlage des Offloadings. *Umgesetzt: Faktor 4.*
2. **Whole-Layer-Offloading.** 15 GB auf 12 GB bei voller Qualität. *Umgesetzt.*
3. **Sparsität auf dem CPU-Offload-Pfad.** *Umgesetzt als opt-in.*
4. **Sparsity-aware Fine-Tuning.** *Induziert g-Sparsität, verbessert aber die ausnutzbare Sparsität nicht (Validierung negativ); korrigierter Ansatz: Strafe auf silu(gate).*
5. **Verlustfreie Kompression** (DFloat11-Ansatz), falls die Kernel-Vorgabe gelockert wird.
6. **MoE-Pfad evaluieren.**

7 Zusammenfassung

● Eigener Beitrag

In einfachen Worten, ohne Fachbegriffe, was dieses Projekt gemessen und gebaut hat.

Was gemessen wurde. Fast die gesamte Rechenarbeit eines Sprachmodells steckt in einem Teil, dem FFN (rund 87 Prozent). Pro Wort arbeiten dort nur wenige der tausenden Neuronen wirklich mit, rund ein Drittel; der Rest trägt fast nichts bei. Die Hälfte davon lässt sich weglassen, und es kommt Wort für Wort dasselbe heraus.

Was gebaut wurde und funktioniert. Eine eigene Programm-Engine, die Sprachmodelle ausführt und gegen die etablierte Referenz auf die Nachkommastelle genau geprüft ist:

- Sie schreibt rund 1,7-mal schneller und gibt garantiert denselben Text aus.
- Ein Modell, das zu groß für die Grafikkarte ist, läuft trotzdem in voller Qualität, nur langsamer.
- Die Berechnung auf dem Prozessor wurde viermal schneller.
- Spart Ressourcen, indem sie schwache Neuronen überspringt.

Was offen ist. Keines dieser Verfahren ist neu erfunden; alle sind bekannt. Die naheliegende eigene Idee, das Modell auf sein Fachgebiet zu spezialisieren, damit es sparsamer rechnet, hat die Messung widerlegt. Eigen ist die vollständige, selbst gebaute und nachgeprüfte Umsetzung auf schwacher Hardware, die als offene Plattform für weitere Experimente dient; fremde, fertige Programme sind geschlossen und erlauben das nicht.

Nächster Schritt. Die Struktur, die noch mehr Tempo bringen würde, ist im Modell natürlich nicht vorhanden. Sie lässt sich möglicherweise durch gezieltes Nachtrainieren erzeugen. Ob das gelingt, ist offen und wird gerade untersucht.

8 Quellen

🔴 Stand der Technik

- Kompression: DFloat11, DFloat11 Code, Shannon-Bound
- Sparsität: TEAL, SparseInfer, Polar Sparsity
- Offloading: PowerInfer, PIPO, NEO
- Spekulatives Decoding: EAGLE-3, Survey
- KV-Cache: PyramidKV, ZigZagKV
- Adaptive Tiefe: Mixture-of-Recursions
- MoE: MoE-Infinity, DALI
- Domäne: Diet Your LLM, Half the Experts, Domain-Specific Expert Pruning